

stm32f103 workspace notes

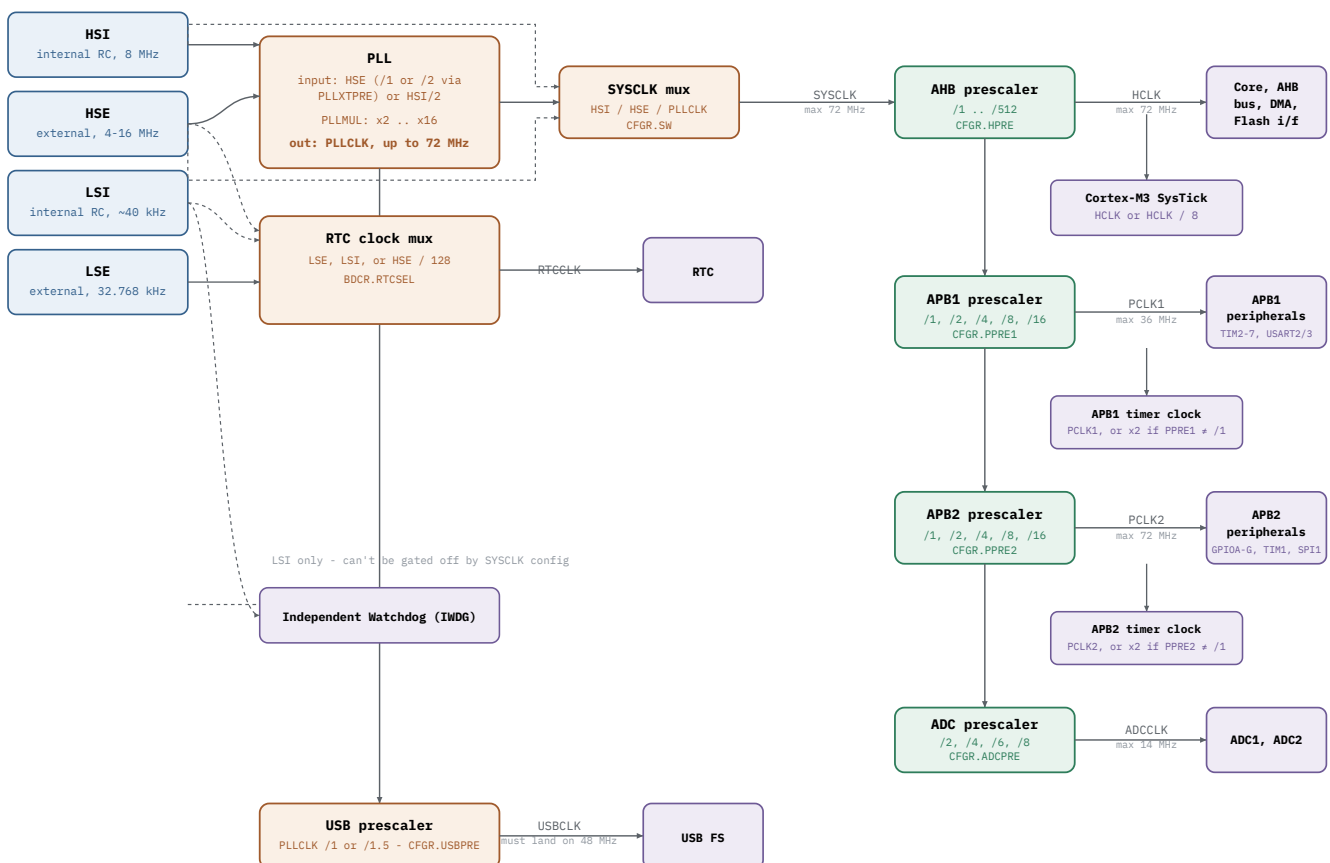
STM32F103 Clock Tree - Reference

Every clock signal on the chip traces back to one of four sources, through at most one multiplier and a couple of dividers. This is the whole tree in one diagram, followed by the handful of configurations actually worth knowing.

oscillators - the four possible sources PLL - the only multiplier on the chip
buses & prescalers where each clock actually goes

The full tree

Original diagram, redrawn from the register-level facts in RM0008 §7 - not a reproduction of the manual's own figure.



Two branches never go through the SYSCLK mux at all: LSI feeds the independent watchdog directly, and LSE/LSI/HSE÷128 feed the RTC through their own separate mux - both keep working even if the main clock configuration is wrong.

What each block actually does

BLOCK	REGISTER	NOTES
HSE enable / ready	<code>RCC_CR.HSEON</code> / <code>HSERDY</code>	Startup isn't instant - always poll <code>HSERDY</code> , ideally with a timeout in case no crystal is fitted
PLL source + multiplier	<code>RCC_CFGR.PLLSRC</code> / <code>PLLXTPRE</code> / <code>PLLMUL</code>	Only writable while <code>RCC_CR.PLLON</code> is 0 - configure fully, then enable
PLL enable / ready	<code>RCC_CR.PLLON</code> / <code>PLLRDY</code>	Also not instant - the PLL needs time to lock
System clock switch	<code>RCC_CFGR.SW</code> / <code>SWS</code>	<code>SW</code> is the request, <code>SWS</code> is hardware's confirmation it actually happened - poll <code>SWS</code> , not just write <code>SW</code> and move on
Flash wait states	<code>FLASH_ACR.LATENCY</code>	Must be set for the <i>target</i> frequency before switching <code>SYSCLK</code> to it - see the table below

Flash latency vs. `SYSCLK` (RM0008)

<code>SYSCLK</code> RANGE	WAIT STATES
0 - 24 MHz	0
24 - 48 MHz	1
48 - 72 MHz	2

Why this bites people. Set the wait states *after* switching to 72MHz and the very next instruction fetch can already be too fast for flash to keep up - the fix is always latency first, frequency second, exactly the order used in `clock_setup_hse_pll_72mhz()` in this workspace's `three-thread-blink_hse` project.

Configuration options that actually get used

Of all the combinations the tree above allows, a handful cover nearly every real project.

CONFIGURATION	SYSCLK	AHB/APB1/APB2	FLASH WS	WHEN TO USE IT
HSI direct no PLL, no crystal	8 MHz	8 / 8 / 8 MHz	0	Default at reset. Fine for bring-up, or anything not timing-sensitive. What every project in this workspace uses except one.
HSE direct no PLL, crystal fitted	= crystal freq (e.g. 8 MHz)	matches SYSCLK	0 (at 8 MHz)	Same speed as HSI but $\pm 1\%$ accuracy instead of HSI's looser tolerance - useful when timing correctness matters more than raw speed.
HSI + PLL no crystal needed	up to 64 MHz	e.g. 64 / 32 / 64 MHz	2	Full speed on boards with no HSE crystal fitted at all. Input is fixed at $\text{HSI}/2 = 4$ MHz, so the multiplier alone sets the ceiling ($\times 16 \rightarrow 64$ MHz).
HSE + PLL the common "full speed" setup	72 MHz	72 / 36 / 72 MHz	2	Maximum rated speed, HSE accuracy. $8 \text{ MHz} \times 9 = 72 \text{ MHz}$ - exactly <code>clock_setup_hse_pll_72mhz()</code> in <code>three-thread-blink_hse</code> .
HSE + PLL, USB-capable	72 MHz	72 / 36 / 72 MHz	2	Same as above, plus <code>USBPRE</code> set to <code>/1.5</code> so <code>PLLCLK</code> (72 MHz) divides down to exactly 48 MHz for the USB peripheral.

Why 72MHz specifically. It's not an arbitrary round number - it's the documented maximum SYSCLK for the whole F103 line, and $8\text{MHz} \times 9$ is the multiplier that lands exactly on it using the crystal nearly every board (including the Bluepill) actually ships with. Boards with a different crystal need a different `PLLMUL` to reach the same ceiling - the target frequency is fixed by the silicon, the multiplier is the variable.

stm32f103 workspace notes · RM0008: STM32F101/102/103/105/107 Reference Manual, §7 (RCC) and Flash programming section